

A Comparative Study of Model-Based and Reinforcement Learning Control Paradigms for Planar Biped Walking

Ritwik Shankar*

Mentor(s): Zach Olkin[†], Dr. Noel Csomay-Shanklin[†], Prof. Aaron Ames[§]

Project Report

California Institute of Technology

Abstract—Choosing between model-based and model-free control strategies for bipedal locomotion is challenging, as each paradigm presents its own set of strengths and limitations. In this work, four distinct strategies are evaluated on a planar biped simulated in NVIDIA’s IsaacSim. First, a pure reinforcement-learning baseline that tracks the commanded base velocity is trained to output joint setpoints for a proportional-derivative (PD) controller. Second, a model-based pipeline employs a reduced-order linear inverted pendulum (LIP) model to generate desired foot trajectories. Using these trajectories, we solve for joint angles via an optimisation-based inverse-kinematics (IK) solver, and apply a PD controller to compute joint torques. Lastly, two hybrid schemes blend learning with physics-based modeling: in the RL-IK-PD approach, a policy proposes foot placements and CusADi (CUDA-accelerated CasADi) with Pinocchio are employed to solve IK using a GPU; in the LIP-RL-PD method, the linear pendulum produces step targets while an RL agent learns to perform the inverse kinematics mapping before PD tracking. The reward ablation shows that the LIP+RL shows more robustness in the variable payload tests, while the baseline-RL performs better on velocity error. We also find that the LIP+RL policy learns optimal walking behaviour in fewer iterations compared to the Baseline-RL policy.

Index Terms—bipedal locomotion, reinforcement learning, inverse kinematics, CasADi, Linear Inverted Pendulum

I. BACKGROUND

A. Introduction

Bipedal locomotion represents a pivotal frontier in legged robotics, offering the promise of human-like agility and the ability to navigate complex, unstructured environments. However, realizing stable two-legged walking is fundamentally challenging due to the system’s underactuated, hybrid dynamics and the intermittent nature of ground contacts. Controllers must manage rapid gait transitions, maintain balance in the presence of disturbances, and compensate for modelling errors and sensory noise. These robots try to follow the commanded base velocity while continuously balancing a high center of mass (CoM) over a small support polygon. To

stay upright they must regulate their CoM and zero-moment point (ZMP) on every step, cope with under-actuated swing phases, and absorb impacts at heel-strike that tightly couple kinematics and whole-body dynamics. Gait stability depends on precise foot placement and timely push-off, and today’s controllers—whether classic model-based schemes such as Linear Inverted Pendulum (LIP) or newer reinforcement-learning (RL) policies—have shown promise in terms of their performance, but still, we are a little far from fully autonomous walking in unknown terrains. Performance gets worse when the terrain isn’t flat, as the controller needs to be very agile to place its foot at a stable location while maintaining the desired velocity.

Some Model-based approaches may provide analytical guarantees (albeit with unreasonable assumptions) but often struggle with robustness and scalability, especially for humanoids, which have a very complicated dynamics with a high number of states and actuators. It is often not possible to solve the Kinematics of a system where the controller is running at a high enough frequency, which could guarantee agility and robustness. Whereas model-free reinforcement learning can yield adaptive policies at the expense of interpretability and sample efficiency. Bridging these paradigms through hybrid methods—combining the structure of classical models with the flexibility of learning—holds significant potential for advancing reliable and efficient bipedal control. These “model-guided” policies have the potential to outperform baseline model-free policies if leveraged correctly.

In this work, we benchmark and dissect four control pipelines ranging from purely model-based controllers to model-free learning-based controllers. We have chosen a planar biped, “AMBER”, as a test bench for our work while using NVIDIA’s IsaacSim for training and simulations. We will start with a purely model-free baseline, training a PPO agent to output PD joint set-points that track commanded forward velocity. I will then build a deterministic, model-based controller that couples a linear inverted-pendulum (LIP) footstep planner with an inverse-kinematics solver, closing the loop with PD torque control. To bridge the gap between these extremes, two hybrids are investigated: (i) **RL-IK-PD** (mentioned as Baseline-RL), where an RL policy proposes

*Undergraduate Student, Department of Aerospace Engineering, Indian Institute of Technology Kanpur

[†]Graduate Students, Department of Control and Dynamical Systems, California Institute of Technology

[§]Bren Professor of Mechanical and Civil Engineering, Control and Dynamical Systems and Aerospace, California Institute of Technology

next footholds that CusADi converts to joint angles for PD tracking; and (ii) **LIP-RL-PD** (mentioned as LIP+RL), where LIP supplies the step targets while an RL network learns the IK mapping itself. Each framework will be trained or tuned under identical terrain and sensor conditions, followed by a systematic reward-function ablation to isolate which terms truly drive stable gaits. Another goal of this project would be to utilise the novel CusADi [1], a GPU-accelerated version of CasADi, which is a symbolic framework to solve optimisation problems. With Pinocchio [2], it can be used to solve the kinematics of a system with just the information of its Unified Robot Description Format (URDF) file and end-effector pose information of the bodies.

B. Literature Review

My primary source of reference for integrating the hybrid method is the work done by Lee et al. [3], which demonstrates a lower-order LIP model for generating future reference foot positions and trains a policy so the robot can follow that path. Vu Nhat Minh [4] provides important insights on planar biped walking and uses the Virtual Pivot Point for the controller. Salman et al. [5] uses three linear pendulums to describe stance, swing and torso dynamics of the bipedal robot. [6]–[8] provides important insights on the implementation of a LIP model as a reduced-order model for a planar biped. Hof et al. [9] gives information about the extrapolated CoM technique that enables to formulate the requirements for stable walking in a relatively simple form

II. CUSADI-BASED INVERSE-KINEMATICS

A. CusADi Integration and setup

1) *Setup*: The first goal was to analyse the applicability of CusADi [1], which is a GPU-accelerated framework of CasADi, which is an optimisation framework. CusADi extends the CasADi symbolic framework to automatically parallelise arbitrary closed-form expressions, on CUDA-enabled GPUs. The Symbolic Expressions (SX) data type is used to represent matrices whose elements consist of symbolic expressions made up by a sequence of unary and binary operations. It parses CasADi expression graphs into sequences of atomic operations and vectorises them, generating custom CUDA kernels capable of evaluating thousands of instances in parallel and achieving up to a ten-fold speedup over CPU implementations [1]. Moreover, CusADi formulates a closed-form approximation for general optimal control problems, enabling large-scale parallel evaluation of model predictive control (MPC) within RL environments [1].

The first subtask was to build CusADi on my system and use it to solve simple Quadratic Problem (QP) and IK problems. I solved nominal Linear Inverted Pendulum Kinematics (converging on to the solution from the differential equation) using CusADi, which requires wrapping the full dynamics into a .casadi function, compiling it with CusADi and then calling it in batches to run in parallel. I ran for N number of parallel solvers and compared the time to solving using classical CasADi, i.e. using a for loop. As you can see in Table

I, even though CPU time increases linearly, the GPU time starts to significantly outperform CPU times as N continues to grow. We can also see that since the GPU solves in batches, the time increase will be discrete, as shown in (Fig. 1).

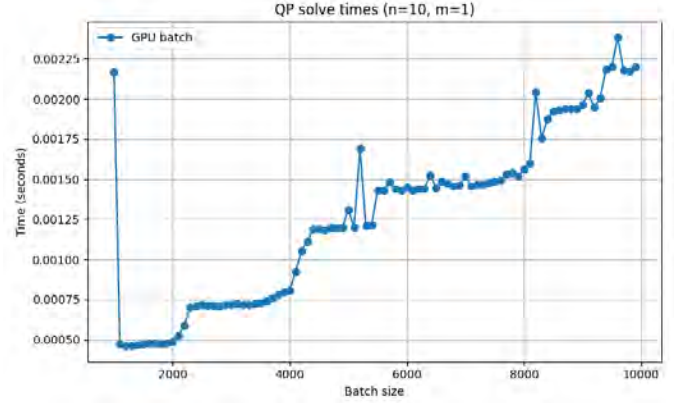


Fig. 1: GPU batch timing vs. sample size N .

TABLE I: Parallel Pendulum Dynamics: Accuracy and Timing

N	Max Abs Error (GPU)	GPU Time (s)	CPU Time (s)
1000	1.137×10^{-13}	0.00250	0.08742
10000	2.842×10^{-14}	0.00248	0.87139
100000	4.547×10^{-13}	0.00270	8.88266
1000000	7.276×10^{-12}	0.00418	86.2259

2) *Testing CusADi for IK*: After successfully utilising CusADi to solve basic Inverse Kinematics. I was given a URDF file for the Go2 robot (Fig. 2.). The task was to generate reference gaits for the robot for a particular combination of (V_x, V_y, w_z) i.e. x linear velocity, y linear velocity and angular velocity about the z axis (yaw) respectively. Since the robot will work for multiple velocities, we need to generate trajectories for an action space (a discrete 3D command range with V_x, V_y & w_x as the three values). In our case, it has 11 discrete V_x , 7 discrete V_y and 5 discrete w_x , which is solving for 385 discrete command profiles. For a particular velocity profile, you will have to generate the foot trajectory of the 4 feet of the robot, which will move with a period of T_s (0.4 seconds in our case, which is half of *PERIOD*). We break this time period into 100 discrete points to give discrete foot positions which connect the initial foot position to the final position via a bezier curve which follows the period and commands the feet to move in a way so that it can follow the commanded base velocity. Solving for 1 foot trajectory (out of the 100 discrete points) requires solving the Inverse Kinematics of the robot to calculate joint angles (Hip, Thigh and Calf) for a particular foot position relative to the CoM.

3) *Generating reference trajectories*: My goal was to first create a .casadi solver, which will use Pinocchio to build the model from its URDF file and solve the IK for the robot using a gradient-based method, then use that IK .casadi function using CusADi to see how well it parallelises over GPUs. Pinocchio is a powerful C++ library with Python bindings,

designed for efficient computation of rigid body dynamics. Via Pinocchio, we use the URDF file and a gradient-based method to converge to the IK solution. Parallelising it over a GPU could have a huge benefit in cases where we are training thousands of robots in parallel and we need to solve the IK of each individual system for its controller. So I compiled my .casadi function with cusADi on CUDA, and called it in parallel in batches to solve for IK. Where a CPU solution of one full velocity profile took around $\sim 16\text{ms}$, a GPU could provide the solution within $\sim 2\text{ms}$, giving an average speedup of around 8x. Even though data transfer is quite slow on a GPU, so the speedup won't be a lot for one instance of IK solve, but to do that for massive parallel simulations, for instance when you are training a policy on 4096 robots, this parallelisation would help you train much faster.

This allowed me to attain two things before I could get into my main work:

- This process helped me to gain insights on how to solve IK of any robot by using its URDF, and to parallelise that process for more effective use.
- This process helped me visualise the reference trajectories I made for my robot, this was a sanity check for my IK solver and my algorithm to generate footsteps to follow a commanded base velocity [10].



Fig. 2: Unitree Go2

B. Formulation of the .casadi IK Function for AMBER

The .casadi function is a CasADi wrapper, which has Pinocchio integrated with it to build the model and solve for the IK while inputting the end-effector positions (footstep location in our case) and the CoM position. The .casadi function needs to be a CasADi SX function (having all SX variables) for it to be parallelisable on CUDA (its a requirement set by developers of CusADi). SX variables in short mean that they will be compiled as a graph; they cannot have any if/else statements, they will be evaluated no matter what,

The .casadi file is essentially a serialised CasADi Function whose body is written entirely with SX-level symbols—CasADi's scalar, acyclic expression type. Within the wrapper we embed Pinocchio's kinematic model so that forward kinematics, frame Jacobians and their algebraic manipulations all appear as SX operations. At run time the function expects only high-level quantities—desired end-effector

poses (here, the foot positions) and current centre-of-mass coordinates—and returns the joint update computed through a damped-least-squares inverse-kinematics routine. Because every quantity involved is an SX object, the function contains no data-dependent branches (if/else, while with unknown bounds, etc.); all control flow is unrolled into pure algebraic expressions. This restriction is mandated by the developers of CusADi, which can launch thousands of IK solves in parallel only when the computational graph is static and branch-free.

Internally, CasADi represents the function as a directed acyclic graph (DAG):

- Nodes correspond to elementary operations (addition, matrix product, trigonometric call, slice, etc.).
- Edges carry symbolic values and capture data dependencies.
- Higher-level constructs such as loops are unrolled, and conditionals are converted into smooth algebraic blends so that every execution follows the same path.

When I save my python CasADi algorithm as .casadi function, CasADi serialises this DAG—including sparsity patterns, operation codes, and literal constants—into a compact, platform-independent file. On load, CusADi parses the graph, schedules it across GPU threads, performs common-sub-expression elimination and fusion, and finally JIT-compiles optimised CUDA kernels. Since the graph is immutable, thousands of IK evaluations across many environments can share the same kernels without recompilation, enabling real-time, massively-parallel footstep tracking and calculation of desired angles of my joints to follow those footsteps. The logic to converge on joint angles using Pinocchio and casadi is given in **Algorithm 1**.

C. Footstep Generation Routine

The algorithm to generate a gait cycle trajectory of the feet (for AMBER), for one Velocity combination, is given in **Algorithm 2**. This function gives us a full gait cycle of the feet, which when followed, commands the robot to follow that specific base velocity combination.

This acts as an important tool for visualisations and plotting our footstep trajectory before implementing it in model-based or learning algorithms. This sanity check helped me to debug my algorithm at various steps, and still helps me generate better swing trajectories for my feet.

D. Visualisation of Open-Loop Footsteps

Show sample plots and discuss limitations of open-loop control.

III. ISAACSIM INTEGRATION

After successfully validating my IK solver and trajectory generation for my robot, It was time to add the robot in IsaacSim.

Algorithm 1 .casadi Inverse Kinematics solver

Require: URDF file $\mathcal{U}$, foot-frame names $\{\text{suppose size } n\}$, damping λ , tolerance ε , iterations N

Require: Inputs at run time: phase ϕ , desired world-frame foot positions $\mathbf{p}_n^w \in \mathbb{R}^{n \times 3}$, COM $(x_{\text{com}}, y_{\text{com}}, z_{\text{com}})$, current joint angles $\mathbf{q}^0 \in \mathbb{R}^m$

Ensure: Joint reference $\mathbf{q}^*$, body-frame foot vector $\mathbf{p}_n^b \in \mathbb{R}^{2n}$, actuated joints m

- 1: Build numeric Pinocchio model from $\mathcal{U}$; cache foot frame IDs $\mathcal{F}_i$
- 2: Convert model to CasADi (cpin) representation for symbolic graph construction
- 3: $\mathbf{p}^w \leftarrow \text{reshape}(\mathbf{p}_{L,R}^w, n \times 3)$
- 4: **for** $i \in \{n\}$ **do** ▷ compute body-frame targets
- 5: $\mathbf{p}_i^b \leftarrow \mathbf{p}_i^w - [x_{\text{com}}, y_{\text{com}}, z_{\text{com}}]^\top$
- 6: **end for**
- 7: $\mathbf{p}_n^b \leftarrow \text{flatten}(\mathbf{p}_0^b, \mathbf{p}_1^b, \dots, \mathbf{p}_n^b)$
- 8: $\mathbf{q} \leftarrow \mathbf{q}^0$
- 9: **for** $k \in \{n\}$ **do** ▷ Damped-least-squares IK per foot
- 10: **for** $j = 1$ **to** N **do**
- 11: FORWARDKINEMATICS($\mathbf{q}$); UPDATEFRAMEPLACEMENTS
- 12: $\mathbf{e} \leftarrow \mathbf{p}_k^b - \mathbf{p}_k^{\text{cur}}$ ▷ position error
- 13: **if** $\|\mathbf{e}\| < \varepsilon$ **then break**
- 14: **end if**
- 15: $\mathbf{J} \leftarrow$ 3-row position part of frame Jacobian at $\mathcal{F}_k$
- 16: $\mathbf{J}^+ \leftarrow \mathbf{J}^\top (\mathbf{J}\mathbf{J}^\top + \lambda \mathbf{I})^{-1}$ ▷ DLS pseudo-inverse
- 17: $\Delta \mathbf{q} \leftarrow \mathbf{J}^+ \mathbf{e}$
- 18: $\mathbf{q} \leftarrow \mathbf{q} + \Delta \mathbf{q}$
- 19: **end for**
- 20: **end for**
- 21: **return** $\mathbf{q}^* \leftarrow \mathbf{q}$, $\mathbf{p}_n^b$

A. URDF Import and Environment Setup

Isaac Sim treats Pixar’s Universal Scene Description (USD) as its native scene graph, so every robot must ultimately be represented as a USD layer before it can be simulated. The workflow starts with the URDF Importer extension (or the CLI-friendly UrdfConverter in Isaac Lab) that parses a URDF file, reads all links, joints, collision meshes and inertias, and then instantiates an equivalent articulated hierarchy inside Omniverse. During conversion the tool embeds each mesh as a USD Geom prim, attaches PhysX rigid-body and joint schemas, and writes articulation metadata (drive type, limits, damping, stiffness) so the robot is immediately “physics-ready.” The resulting file—typically saved as $\langle \text{robot} \rangle .\text{usd}$ —becomes a reusable asset that can be referenced into any stage, benefit from USD’s powerful layering, instancing and variant mechanisms, and load in milliseconds because only the composed layer graph is streamed, not the entire mesh payload. In short, the URDF step provides human-legible kinematics, while USD provides the high-performance, version-controllable format that Isaac Sim’s rendering, PhysX backend and ROS bridges all expect.

Algorithm 2 GENERATEREFERENCETRAJECTORY

Require: planar forward speed v_x , foot-frame IDs $\mathcal{F}$, default foot positions $\mathbf{P}^{\text{def}}$, swing height h_{sw} , cycle period T , samples N

Ensure: time vector $\mathbf{t} \in \mathbb{R}^N$, joint reference table $\mathbf{Q}^* \in \mathbb{R}^{N \times 4}$, body-frame foot table $\mathbf{P}^b \in \mathbb{R}^{N \times 2 \times 3}$

- 1: **Pre-allocate** CasADi DMs: $\mathbf{Q}^* \leftarrow \mathbf{0}_{N \times 4}$, $\mathbf{P}_{\text{flat}}^b \leftarrow \mathbf{0}_{N \times 6}$
- 2: $\mathbf{t} \leftarrow \text{linspace}(0, T, N)$; $\phi \leftarrow \max(0, \min(1, (t/T - 0.25) \cdot 2))$ ▷ phase
- 3: Build **swing-height profile** $\mathbf{z} \in \mathbb{R}^N$ via cubic Bézier:
- 4: $z_i \leftarrow \begin{cases} B(0 \rightarrow h_{\text{sw}}, 2\phi_i) & \phi_i \leq 0.5 \\ B(h_{\text{sw}} \rightarrow 0, 2\phi_i - 1) & \text{otherwise} \end{cases}$
- 5: $\mathbf{x}_{\text{com}} \leftarrow v_x \mathbf{t}$; $\mathbf{y}_{\text{com}} \leftarrow \mathbf{0}$ ▷ planar COM path
- 6: Form matrix $\mathbf{P}_{2 \times 3}^{\text{def}}$ from dictionary entries
- 7: $\mathbf{P}_{\text{com}}^0 \leftarrow [x_0, y_0, 0]^\top$; $\mathbf{P}_{\text{com}}^T \leftarrow [x_N, y_N, 0]^\top$
- 8: $\mathbf{P}_0^w \leftarrow \text{repmat}((\mathbf{P}_{\text{com}}^0)^\top, 2, 1) + \mathbf{P}^{\text{def}}$
- 9: $\mathbf{P}_T^w \leftarrow \text{repmat}((\mathbf{P}_{\text{com}}^T)^\top, 2, 1) + \mathbf{P}^{\text{def}}$
- 10: **For** each time index $i = 1 \dots N$ **do**
- 11: $\mathbf{P}_i^w \leftarrow B(\mathbf{P}_0^w, \mathbf{P}_T^w, \phi_i)$
- 12: $\mathbf{P}_i^w(L_z) += z_i$, $\mathbf{P}_i^w(R_z) += z_i$ ▷ add vertical swing
- 13:
- 14: $\mathbf{q}_{\text{prev}} \leftarrow \mathbf{q}_{\text{init}}$
- 15: **for** $i = 1$ **to** N **do** ▷ call compiled IK
- 16: q_i , $\mathbf{p}_i^b \leftarrow \text{AMBERIKSOLVER}(\mathbf{P}_i^w, \text{com}_i, \mathbf{q}_{\text{guess}} = \mathbf{q}_{\text{prev}})$
- 17: $\mathbf{Q}^*[i] \leftarrow q_i$; $\mathbf{P}_{\text{flat}}^b[i] \leftarrow \text{vec}(\mathbf{p}_i^b)$
- 18: $\mathbf{q}_{\text{prev}} \leftarrow q_i$
- 19: **end for**
- 20: Convert $\mathbf{Q}^*$ and $\mathbf{P}_{\text{flat}}^b$ to NumPy; reshape $\mathbf{P}^b$ to $N \times 2 \times 3$
- 21: **return** $\mathbf{t}$, $\mathbf{Q}^*$, $\mathbf{P}^b$

Although the tool is quite effective, I still had to manually add configurations like Angular Limits, Stiffness, Damping, Planar Constraints, etc and features like inertias for my bodies, as well as specify the root links and end-effector positions of my robot.

B. ArticulationCfg and Constraints

My Isaac Sim implementation represents AMBER as a single ArticulationCfg layer. The **spawn** block references the converted `amber.usd` asset and turns on PhysX contact sensors so that foot-ground impulses can be queried at every step. Rigid-body properties disable neither gravity nor gyroscopic forces and raise the linear/angular velocity caps to 10^3 m s^{-1} and 10^3 rad s^{-1} , respectively, to avoid artificial clamping during flight phases. The articulation-root block follows NVIDIA’s recommended solver settings—eight position and four velocity iterations—while self-collision is disabled because the torso and thigh volumes never intersect in a strictly planar gait.

Initial joint angles are zeroed and the base link is spawned at $(0, 0, 1.7) \text{ m}$, well above the rail surface, so that the controller experiences a short drop before the first contact event. A soft-limit factor of 0.9 is applied to every revolute joint to



Fig. 3: Massively parallel simulation env (4096 robots)

allocate a small buffer outside the CAD limits for numerical drift.

Each of the four actuated joints (namely “q1_right”, “q2_right”, “q2_left” and “q2_left”, where q1 is thigh and q2 is calf) is driven by an **ImplicitActuatorCfg** with identical stiffness ($K = 1000 \text{ N m rad}^{-1}$) and damping ($D = 50 \text{ N m s rad}^{-1}$); effort limits are set to 400 N m , matching the motor data sheet.

1) *Unstable Simulation due to constraints*: Early experiments added hard six-degree-of-freedom (DoF) constraints between the base link and world, making the robot strictly allow translation only in X & Z axes and rotation about the Y axis. This proved to be a big mistake: the redundant positional targets over-constrained PhysX, causing the solver to generate NaN velocities that propagated into NaN rewards and sent the PPO training into numerical meltdown. I spent nearly a week instrumenting dozens of nested variables and plotting traces before tracing the instability back to my attempt at hard-wiring planar motion. A further drawback emerged when I tried to spawn robots in a 10×10 grid: any unit whose initial y -position was non-zero was forcibly snapped to $y = 0$, because the constraint demanded an invariant lateral coordinate.

2) *Prismatic rail constraints*: Inspired by Isaac Lab’s cart-pole example, I replaced the six DoF constraints with two orthogonal *prismatic* rails: one along x for forward motion and one along z for vertical oscillation, for the pitch motion, I added a 1DOF free rotation joint, joining the end of articulation of the rails and the start of the articulation of the robot (torso). Roll, yaw and lateral y motion are thus passively locked by the rail geometry rather than by conflicting joint drives. This change eliminated NaN bursts and made my PhysX simulation completely stable. The setup with rails can be seen in Fig. 3. The Isaacsim USD configuration can be seen in Fig. 4. You can see the 2 rails to provide planar movements, a “Cube” to provide pitch movement, which is fixed to the “torso”. The “Capsule” you see in “torso” is a collision root added by me to detect any fall-based collisions for resetting that particular environment. “right_toe_sphere” and “left_toe_sphere” are fixed bodies to their respective foot added to provide the end-effector position of those joints as well as detect collision of the foot.

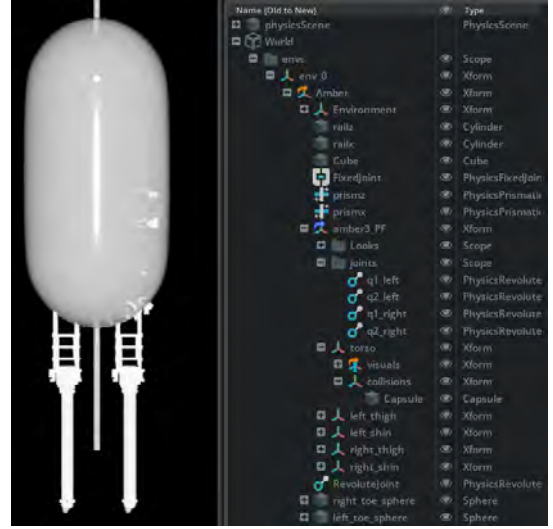


Fig. 4: Amber USD configuration

IV. REINFORCEMENT-LEARNING BASELINE

Once Amber was fully integrated into Isaac Sim and running stably, I ran basic sanity tests—commanding random joint actions—to confirm that the robot responded with the expected motions and that the contact-based fall-termination logic triggered correctly. The next step was to start training a “baseline” Reinforcement learning policy for walking. I had to generate multiple configuration files to set up my ManagerBasedRL Isaacsim environment. All the code files can be found on this clickable GitHub link. (I have mentioned some files from this repo)

1) *Config inheritance chain.*: Fig. 5 summarises the four-level hierarchy (source code):

- 1) **AmberFlatEnvCfg** (amber_flat_env_cfg.py): disables height scanning and forces the terrain generator to a simple plane.
- 2) **AmberRoughEnvCfg** (amber_rough_env_cfg.py): adds procedurally generated stairs, bumps and friction randomisation; inserts three contact sensors and re-weights the reward terms for rough terrain.
- 3) **AmberEnvCfg** (amber_env_cfg.py): declares the *MDP blocks* — actions, observations, rewards, events — and plugs in the robot asset (AMBER_CFG).
- 4) **LocomotionVelocityRoughEnvCfg** (Isaac Lab base): provides the ManagerBasedRLEnv boilerplate, including command generators, terrain curriculum and default termination rules.

2) *ManagerBasedRLEnv specifics.*: At runtime the ManagerBasedRLEnv created by the chain above instances four managers:

- **Action Manager** — maps the policy’s (size 4) output to joint position targets (JointPositionActionCfg).
- **Observation Manager** — collates observation terms (phase clock, projected gravity, joint states, etc.) with independent history length and noise models.

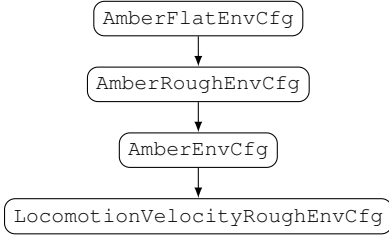


Fig. 5: Config inheritance used by `train.py`.

- **Reward Manager** — evaluates 20+ shaped rewards, including forward-velocity tracking, symmetry metrics and foot-slip penalties.
- **Event / Termination Managers** — injects terrain pushes, resets fallen robots and ends an episode when torso contact forces exceed 1 N.

Because all four managers are configured declaratively inside `AmberEnvCfg`, moving from the *flat* environment to the *rough* variant requires no changes to `train.py`; one simply launches

```
python train.py --task amber-rough-vel
```

and Hydra skips loading the `AmberFlatEnvCfg`, which disables rough environment settings and directly loads `AmberRoughEnvCfg`, which sets up the reward weights and termination-based logic.

A. Action and Observation Spaces

1) *Policy observations*: Table II lists the sensors supplied to the network, together with the length of each contribution in the concatenated observation vector. In total, 21 Observations are given to the policy.

Observation	Dim.	Purpose / Description
base_ang_vel	3	World-frame angular velocity of the torso; helps damp pitch oscillations
projected_gravity	3	Gravity vector expressed in the base frame; compact orientation cue
velocity_commands	1	Desired forward speed command (planar v_x) the robot should track
joint_vel	4	Angular velocities of the four actuated joints, normalised and noise-perturbed
joint_pos	4	Joint angles relative to a neutral stance; gives absolute posture info
actions	4	Previous actions (PD setpoints) given by the policy in the previous timestep
sin_phase, cos_phase	1 + 1	Phase clock signals (sin and cos) that provide an explicit timing reference for gait scheduling

TABLE II: Observation terms and sizes used by the Amber policy

2) *Critic observations*: The critic additionally receives a 3-D `base_lin_vel` term, which contains the base velocity of the torso in the world frame, making the total observations for the critic network to be 24.

3) *Action space*: The policy outputs a four-dimensional vector of joint-position targets for `q1_left`, `q2_left`, `q1_right`, and `q2_right`. These targets are converted to

torques by an `ImplicitActuatorCfg` with fixed gains $K_p = 1000 \text{ Nm rad}^{-1}$ and $K_d = 50 \text{ Nms rad}^{-1}$

B. Policy Architecture and PPO Setup

1) *Network topology*: Both actor and critic share the same multilayer perceptron layout: three fully connected layers of $\{512, 256, 128\}$ ELU units, followed by a linear output head. The input is a 29D vector (observations) and the output is a 4D vector (actions). Weights are initialised with unit-variance Gaussian noise (`init_noise_std=1.0`) to encourage early exploration; the log- standard-deviation of the Gaussian action distribution is fixed during training (i.e. `learn_std=False`). Table III lists the principal hyper-parameters.

Runner		PPO algorithm	
Steps / env	24	Clip ratio	0.20
Mini-batches	4	Entropy coef.	0.008
Epochs / iter	5	Value-loss coef.	1.0
Learning rate	1×10^{-3}	γ (discount)	0.99
Max grad-norm	1.0	λ (GAE)	0.95
Desired KL	0.01	Schedule	adaptive
Save every	50 iters		

TABLE III: Key PPO hyper-parameter

2) *Proximal Policy Optimisation*: I am implementing Isaac Lab’s PPO algorithm. PPO is an on-policy actor–critic method that alternates between data collection and gradient updates. During each iteration, the runner executes the current policy for $N_{\text{env}} \times 24$ steps, storing states, actions, rewards and value predictions. Generalised Advantage Estimation (GAE) converts these rollouts into advantage targets A_t while the critic is trained to regress discounted returns. The actor is updated by maximising the *clipped surrogate* objective

$$L^{\text{CLIP}} = \mathbb{E}[\min(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)],$$

where $r_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{\theta_{\text{old}}}(a_t | s_t)$ and $\epsilon = 0.2$. Clipping restricts the policy update to a “trust region” so that large advantage estimates cannot drive the new policy far from the behaviour policy. An adaptive KL schedule reduces the learning rate whenever the measured KL divergence exceeds 0.01, keeping successive policies *proximal* and stabilising training.

The main PPO hyperparameters tune the balance between exploration, update stability, and value-function accuracy. Hyperparameters used for my training case are given as follows:

- **Clip ratio** ($\epsilon = 0.2$) – bounds the likelihood ratio $r_t(\theta)$ so that each policy update stays within a small “trust region”; lower values are safer, higher values learn faster but risk divergence.
- **Entropy coefficient** (0.008) – adds $-\beta H[\pi]$ to the loss, encouraging higher-entropy action distributions and sustained exploration.
- **Value-loss coefficient** (1.0) and *clipped value loss* – weight the critic’s mean-squared-error term and clip its updates, keeping value estimates in step with the actor.

- **Discount factor** ($\gamma = 0.99$) – sets an effective planning horizon of ~ 100 steps ($1/(1-\gamma)$), appropriate for 0.8-s gait cycles.
- **GAE decay** ($\lambda = 0.95$) – trades bias for variance in Generalised Advantage Estimation; larger λ uses longer returns, smaller values rely on one-step TD errors.
- **Desired KL** (0.01) – target KL divergence for the adaptive learning-rate scheduler; when the measured KL exceeds this threshold, step sizes are reduced to keep successive policies *proximal*.
- **Max gradient norm** (1.0) – global gradient clipping that prevents exploding updates, especially after large contact rewards or penalties.

Together, these terms let PPO make steady, sample-efficient progress while maintaining numerical stability in the contact-rich locomotion domain. tabularx

V. REWARD ENGINEERING

Planar bipeds attracted most of their research attention during the 2000s and early 2010s, well before reinforcement learning surged in popularity in the late-2010s. As a result, the literature on applying modern RL techniques to planar walkers is relatively sparse. Moreover, designing effective reward functions proved time-consuming in my own work: the relationship between reward shaping and emergent behaviour is highly non-intuitive, so the final formulation emerged only after extensive trial-and-error. The reward functions formulated are as follows in Table IV.

VI. STANDALONE MODEL-BASED CONTROLLER

Bipedal locomotion is often modeled as a single inverted pendulum connecting the support foot to the center of mass (CoM) via a massless telescopic leg [9]. By constraining the CoM height to be constant and assuming a point-foot contact (no ankle actuation), the planar Linear Inverted Pendulum Model (LIPM) dynamics reduce to

$$\ddot{\mathbf{r}} = \omega_0^2 (\mathbf{r} - \mathbf{p}) \quad (1)$$

where $\mathbf{r} = (r_x, r_y)^T$ is the horizontal CoM position, $\mathbf{p} = (p_x, p_y)^T$ is the foot position on the ground, and

$$\omega_0 = \sqrt{\frac{g}{z_0}}$$

is the natural frequency determined by gravity g and the constant CoM height z_0 .

Integrating (1) yields the system’s “orbital energy” and motivates the definition of the Instantaneous Capture Point (ICP) [11], which is the ground point at which the pendulum would come to rest if the foot were placed there instantaneously:

$$\xi = \mathbf{r} + \frac{\dot{\mathbf{r}}}{\omega_0} \quad (2)$$

Differentiating (2) in time and substituting (1) gives the first-order ICP dynamics:

$$\dot{\xi} = \omega_0 (\xi - \mathbf{p}) \quad (3)$$

The closed-form solution of (3) over a time interval t is

$$\xi(t) = e^{\omega_0 t} \xi(0) + (1 - e^{\omega_0 t}) \mathbf{p} \quad (4)$$

These ICP dynamics form the basis for stable footstep planning: by predicting the evolution of ξ , one can compute where to place the next foot to capture the pendulum (i.e., reestablish balance) under velocity commands [12], [13].

A. Trajectory generation

The compute LIP function is used to get the future foot position setpoint. Then a spline, using a bezier (almost identical to **Algorithm 2**) is used to connect the current swing foot to the future position in N (100 for our case) steps. The controller frequency has to be maintained such that every half cycle when the LIP function is run, there should be exactly 100 iterations of sending these reference trajectories to my IK solver (**Algorithm 1**) where my function converges to the necessary angles and those angles are then sent as setpoints to the robot to follow these trajectories.

VII. LIP-RL-PD PIPELINE

The setup for the LIP-RL pipeline consists of an extra observation term and a reward term so that my policy is trained to follow a desired foot position. **Algorithm 3** consists of the logic of computing the future LIP foot position.

A. Observation

Initially, the policy was supplied with the full six-dimensional desired foot pose and the six-dimensional current foot pose. Training failed to converge due to the scale mismatch between these high-dimensional vectors and the other state inputs. I then reduced the observation to a four-dimensional vector comprising the relative X and Z offsets between the current and target foot positions, which immediately stabilised learning. This modification increased the policy’s input feature count from 21 to 25 and the critic’s from 24 to 28. In future work, I will explore alternative observation sets, such as removing the command velocity and relying solely on footstep targets to isolate how each component influences performance.

B. Reward(s)

Contact-Step Centre-of-Mass Reward:
Rewards or punishes both swing and stance leg on their position in a gait cycle according to the phase, while scaling that factor with the L2 norm error of the current and desired positions (**Algorithm 4**).

TABLE IV: Reward terms used in locomotion training.

Reward Name	Weight	Expression
termination_penalty	-400	$r_t \leftarrow r_t - 400$, if pelvis in contact
track_lin_vel_x	30	$r_t = 120 \exp\left[-\frac{(v_x^{\text{cmd}} - \bar{v}_x)^2}{2\sigma_v^2}\right]$, $\sigma_v = 0.4$
joint_angles	-4	$r_t = -4 \left[\exp\left(\frac{\Delta\theta}{\sigma_\theta}\right) - 1 \right]$, $\Delta\theta = \max(\theta_j - \theta_{\text{th}}, 0)$, $\theta_{\text{th}} = 2^\circ$
joint_symmetry_reward	+1	$r_{\text{hold}} = \begin{cases} 5e^{-d/\sigma}, & d \leq d_{\text{th}} \\ -2.5(d - d_{\text{th}}), & d > d_{\text{th}} \end{cases}$
stride_consistency	-0.8	$r_{\text{hold}} = -\min(\text{stride}_{\text{now}} - \text{stride}_{\text{prev}} , 0.8)$
torso_orientation	+1	$r_t = \begin{cases} +2, & \alpha \leq 4^\circ \\ -\min(e^\alpha, 20), & \alpha > 15^\circ \\ 0, & \text{otherwise} \end{cases}$
alternation_contact	+1	$r_t = \begin{cases} +1, & \text{if alternating (L} \rightarrow \text{R or R} \rightarrow \text{L)} \\ -1, & \text{if repeat contact} \end{cases}$
progressive_step	+1	$r = \begin{cases} \min(\Delta x, 0.2), & \Delta x > 0 \\ -5, & \Delta x \leq 0 \end{cases}$, $r_t = r$
feet_no_slip_condition	-15	$r_t = -15 \ \mathbf{v}_{\text{foot}}\ ^2$, if in contact
phase_reward	+9	$r_t = (L_R - L_L) \sin\left(\frac{2\pi t}{\text{PERIOD}}\right)$
alive	+0.15	constant positive reward each step
lin_vel_z_l2	-0.2	$-\ v_z\ ^2$
ang_vel_xy_l2	-0.05	$-\ \omega_{xy}\ ^2$
dof_torques_l2	-1.0e-5	$-\ \tau\ ^2$
dof_acc_l2	-2.5e-7	$-\ \ddot{q}\ ^2$
dof_pos_limits	-5.0	penalty if joint outside limits
flat_orientation_l2	-1.0	$-\ \text{tilt}\ ^2$

Algorithm 4 Contact-Step Center-of-Mass Reward

Require: $L_r, L_l \in \{0, 1\}$ $\triangleright$ Right/left foot contact events
 1: ϕ_{contact} $\triangleright$ Phase-based contact weighting
 2: $\hat{\mathbf{p}}, \mathbf{p} \in \mathbb{R}^n$ $\triangleright$ Desired and current foothold positions
 3: $\sigma > 0$ $\triangleright$ Distance scale parameter
Ensure: r_{cs} $\triangleright$ Contact-step COM reward
 4: $\Delta L \leftarrow L_r - L_l$ $\triangleright$ Positive when only right foot in contact, negative for left
 5: $d \leftarrow \|\hat{\mathbf{p}} - \mathbf{p}\|_2$ $\triangleright$ Euclidean error between desired and actual foothold
 6: $r_{cs} \leftarrow \Delta L \times \phi_{\text{contact}} \times \exp(-d/\sigma)$
 7: **return** r_{cs}

VIII. RESULTS

The two policies, namely Baseline-RL & LIP+RL, were deployed in simulation to generate insights based on their performance, robustness & sampling efficiency.

A. Performance

An important thing to note while reward-tuning bipedal robots is that the tuning is highly non-intuitive. Defining a reward which can mathematically showcase ideal human-like walking is tricky, since the policy could always hack its way into providing sub-optimal solutions while maximising rewards. Building on this point, it is as hard to define human-like walking behaviour mathematically; we could always use phase-based rewards like a lot of people do, but in uneven terrains, they perform sub-optimally. Hence, the correct way of judging a policy is both qualitatively and quantitatively.

Consider a velocity error-based performance metric (5) or a LIP error-based metric (6)-

$$\text{Velocity Penalty} = \text{Mean} \left[\left| \frac{V_{\text{curr}} - V_{\text{cmd}}}{V_{\text{cmd}}} \right| \times 100 \right] \quad (5)$$

$$\text{LIP Score} = \text{Mean} \left[\left| \frac{Foot_{\text{curr}} - Foot_{\text{LIP}}}{Foot_{\text{LIP}}} \right| \times 100 \right] \quad (6)$$

We came across multiple policies that generated low Velocity Penalty, but consisted of impractical gaits. Hence, adding a qualitative aspect for judgment was also necessary. Figure 6 & Figure 7 show the velocity tracking performance of Baseline-RL & LIP+RL respectively. The former generates a “Velocity Penalty” of ~ 24 while the latter generates ~ 36 . An obvious explanation of LIP+RL not performing up to the mark is that Baseline-RL is tuned to minimise the velocity error, while the LIP+RL is tuned to minimise the LIP foot position error. So even if it’s able to track the LIP positions (Figure 8), it’s performing poorly on velocity, which can be directly linked to non-precise LIP implementation on our part. Figure 9 & 10 consists of 2D & 3D plots of LIP footsteps tracking

B. Robustness

To test & compare the two policies on the basis of robustness, we added the payload mass on the torso (technically, just increased the torso mass from its original 13 kg, in steps of 0.5 kg to failure). Figure 11 & 12 are the variations of individual error when subjected to a payload mass. On average, it was seen that LIP+RL could walk without failure for a higher payload value compared to Baseline-RL. Our “Model-guided” policy could handle this robustness far better than the Baseline version, hinting that crucial information from the model helps

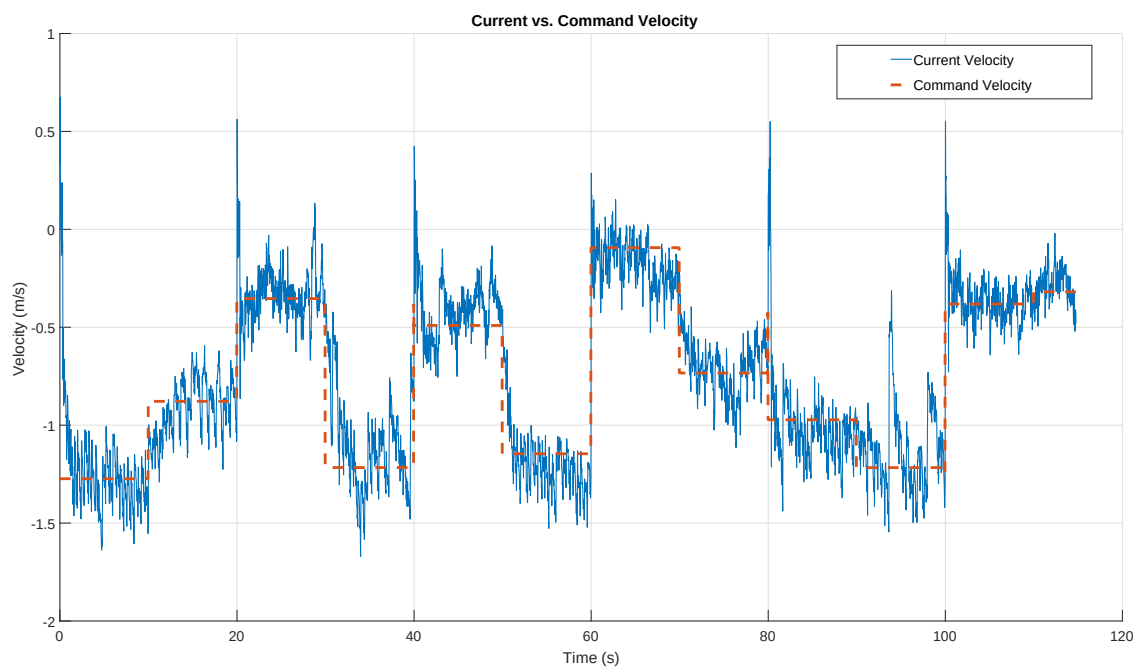


Fig. 6: Velocity tracking of Baseline-RL

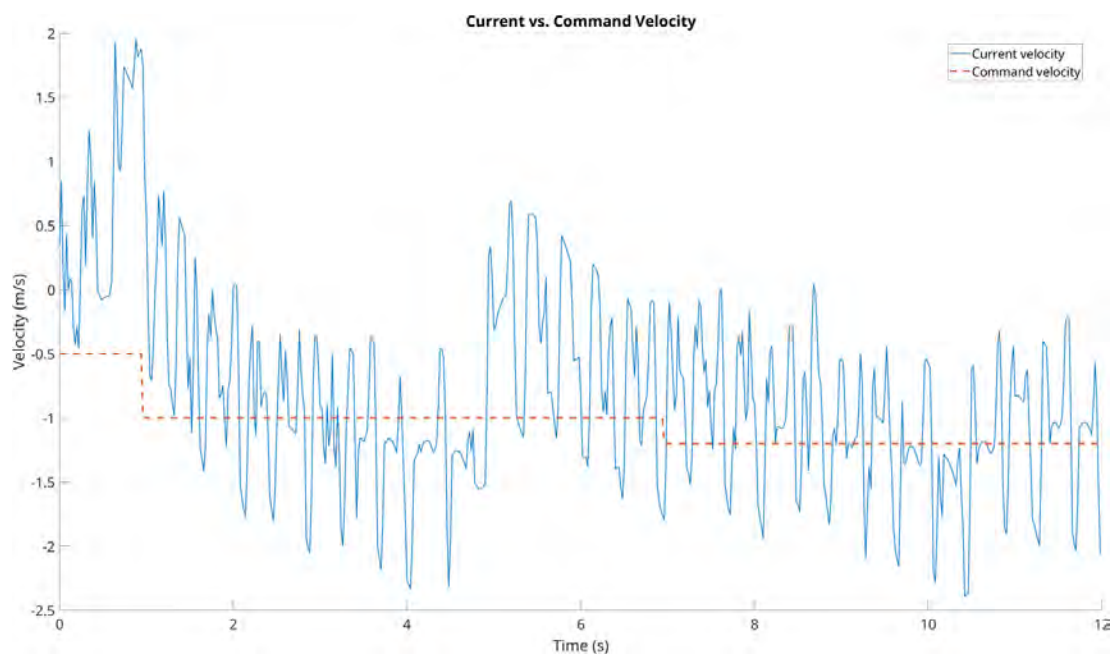


Fig. 7: Velocity tracking of LIP+RL

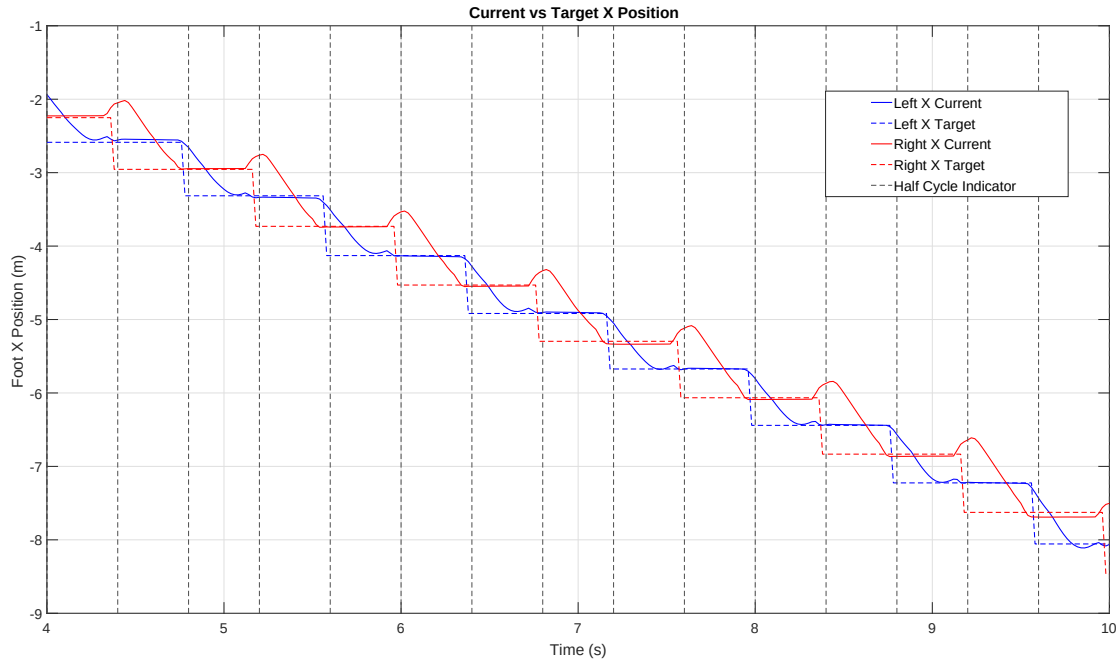


Fig. 8: LIP foot position tracking

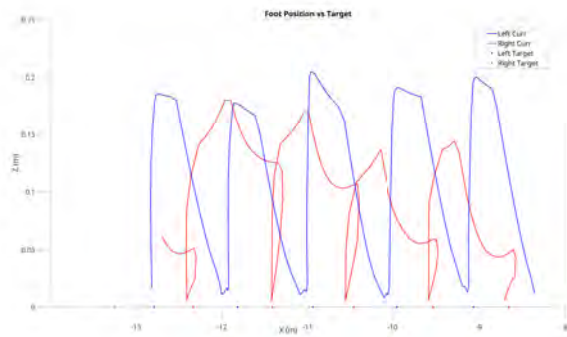


Fig. 9: 2D LIP footsteps in X-Z view

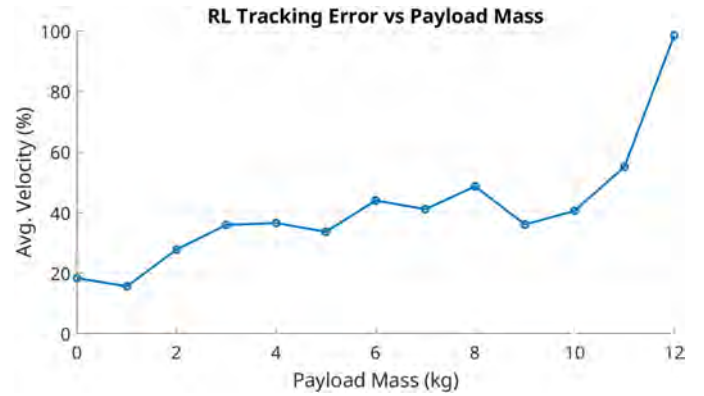


Fig. 11: Baseline-RL Velocity Penalty vs Payload

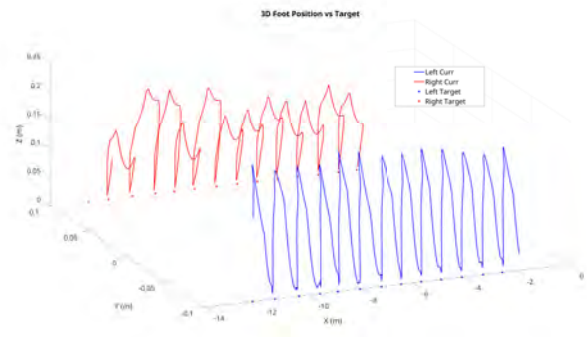


Fig. 10: 3D LIP footsteps in the coordinate frame

our system in staying stable. Qualitative analysis of the two policies shows that 12 Kg payload forces our Baseline-RL policy to hit failure by falling, while the LIP+RL was able to lift up to 20 Kgs of payload before falling.

C. Sampling Efficiency

Overall 50 policies were trained while performing the ablation study. Policies were not only compared on the final error, but also on iteration-wise error. It was found that on average, LIP+RL takes a longer time per iteration as compared to Baseline-RL, but it learns walking behaviour much faster than Baseline-RL in terms of iterations. This knowledge could be utilised in future training and deployment of such bipeds, especially where we need to retrain a policy based on current needs, and we want to do it quickly.

Algorithm 3 Compute Next Foothold via Local-Frame LIP-ICP

Require: sim_time , scene , args_cli , nom_height , T_{swing} , w_{des} , visualize

Ensure: $p \in \mathbb{R}^{7_{\text{envs}} \times 3}$ $\triangleright$ Next footholds in world frame

```

1:  $\text{amber} \leftarrow \text{scene}["\text{Amber}"]$ 
2:  $n_{\text{envs}} \leftarrow \text{args\_cli.num\_envs} \triangleright$  — 1) Cache original lateral Y offsets once —
3: if not  $\text{hasattr}(\text{compute\_step\_location\_local}, \text{"\_orig\_foot\_y"})$  then
4:    $\text{\_orig\_foot\_y} \leftarrow$  extract foot-Y from  $\text{amber.data.body\_pos\_w}$ 
5:   store as static attribute
6: end if
7:  $\omega \leftarrow \sqrt{g/\text{nom\_height}}$   $\triangleright$  — 2) Commanded velocity and COM —
8:  $\text{cmd} \leftarrow \text{args\_cli.desired\_vel}[2]$ 
9:  $\text{icp\_0} \leftarrow \text{cmd}/\omega$ 
10:  $r \leftarrow$  weighted average of  $\text{amber.data.body\_com\_pos\_w}$   $\triangleright$  — 3) Last foot positions and stance selection —
11:  $\text{foot\_pos} \leftarrow \text{amber.data.body\_pos\_w}[:, \{-2, -1\}, :]$ 
12:  $\phi \leftarrow (\sin(2\pi \frac{\text{sim\_time}}{2T})) / \sqrt{\sin^2(\cdot) + T}$ 
13:  $\text{stance\_idx} \leftarrow \begin{cases} 0 & \phi \geq 0 \\ 1 & \text{else} \end{cases}$ 
14:  $\text{stance\_foot} \leftarrow \text{foot\_pos}[:, \text{stance\_idx}, :]$ ; set  $z = 0$ 
15: function TO_LOCAL( $v, q$ )  $\triangleright$  — 4) Local-to-global transforms —
16:   return  $R_{\text{yaw}}(q)^T v$ 
17: end function
18: function TO_GLOBAL( $v, q$ )
19:   return  $R_{\text{yaw}}(q) v$ 
20: end function
21:  $\text{icp\_f} \leftarrow e^{\omega T} \text{icp\_0} + (1 - e^{\omega T}) \text{to\_local}(r - \text{stance\_foot}, q)$   $\triangleright$  — 5) ICP update in local frame —
22:  $\text{icp\_f}[:, 2] \leftarrow 0$ 
23:  $s_d \leftarrow |\text{cmd}_x| T$ ,  $w_d \leftarrow w_{\text{des}}$   $\triangleright$  — 6) Bias and clipping —
24:  $b_x \leftarrow s_d / (e^{\omega T} - 1)$ ,  $b_y \leftarrow \text{sign}(\phi) w_d / (e^{\omega T} + 1)$ 
25:  $p_{\text{local}} \leftarrow \text{icp\_f} - [b_x, b_y, 0]$ 
26:  $\text{clamp } p_{\text{local}}[:, 0] \in [-0.25, 0.25]$ ,  $p_{\text{local}}[:, 1] \in [-0.3, 0.3]$   $\triangleright$  — 7) Back to world frame and enforce robot's Y —
27:  $p \leftarrow \text{to\_global}(p_{\text{local}}, q) + r$ 
28:  $p[:, 2] \leftarrow 0$ 
29:  $p[:, 1] \leftarrow \text{\_orig\_foot\_y}[:, 1 - \text{stance\_idx}]$ 
30: if  $\text{visualize}$  then
31:   draw spheres at  $p$  and COM in USD
32: end if
33:  $\text{scene.current\_des\_step} \leftarrow p$   $\triangleright$  — 8) Store and return —
34: return  $p$ 

```

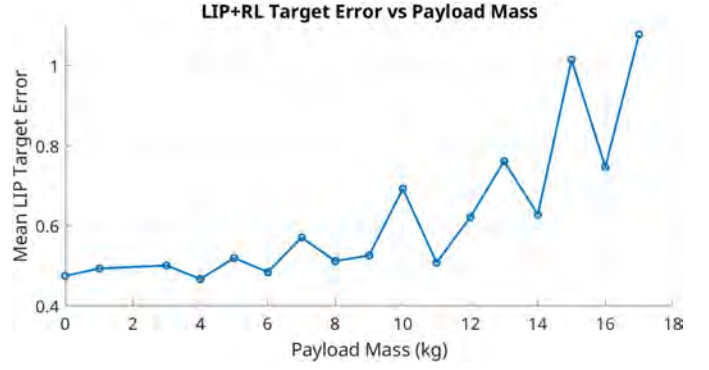


Fig. 12: LIP+RL LIP score vs Payload

IX. CONCLUSION & FUTURE WORK

Currently, we have robust pipelines for both the baseline-RL and the LIP+RL approaches for AMBER in IsaacSim. Building a purely model-based controller is proving more intricate—our LIP assumptions introduced many floating-point parameters that must be managed carefully. The majority of the issues which are making a case against LIP+RL are based in the LIP model implementation. Fixing the model to compare the two policies with a purely model-based classical controller would provide even more insights in this paradigm of choosing between model-free or model-based controllers.

REFERENCES

- [1] S. H. Jeon, S. Hong, H. J. Lee, C. Khazoom, and S. Kim, “Cusadi: A gpu parallelization framework for symbolic expressions and optimal control,” 2024. [Online]. Available: <https://arxiv.org/abs/2408.09662>
- [2] J. Carpentier, G. Sauerl, G. Buondonno, J. Mirabel, F. Lamiriaux, O. Stasse, and N. Mansard, “The pinocchio c++ library – a fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives,” in *IEEE International Symposium on System Integrations (SII)*, 2019.
- [3] H. J. Lee, S. Hong, and S. Kim, “Integrating Model-Based Footstep Planning with Model-Free Reinforcement Learning for Dynamic Legged Locomotion,” 8 2024. [Online]. Available: <https://arxiv.org/abs/2408.02662v1>
- [4] M. N. Vu, “Hierarchical control strategy for planar bipedal walking robots based on reduced order model,” 2 2023. [Online]. Available: <http://arxiv.org/abs/2303.12180>
- [5] S. Faraji and A. J. Ijspeert, “3LP: a linear 3D-walking model including torso and swing dynamics,” 5 2016. [Online]. Available: <http://arxiv.org/abs/1605.03036>
- [6] S. Kajita, F. Kanehiro, K. Kando, K. Yokoi, and H. Hirukawa, “IEEEIRsl International Conference on Intelligent Robots and Systems The 3D Linear Inverted Pendulum Mode: A simple modeling for a biped walking pattern generation,” Tech. Rep.
- [7] H. I. Hemami and C. L. Golliday, “The inverted Pendulum and Biped Stability*,” Tech. Rep.
- [8] X. Xiong and A. Ames, “3D Underactuated Bipedal Walking via H-LIP based Gait Synthesis and Stepping Stabilization,” 11 2021. [Online]. Available: <http://arxiv.org/abs/2101.09588>
- [9] A. L. Hof, “The ‘extrapolated center of mass’ concept suggests a simple control of balance in walking,” *Human Movement Science*, vol. 27, no. 1, pp. 112–125, 2 2008.
- [10] s ritwik, “Sample walking simulation video,” GitHub repository, Available at: https://github.com/s-ritwik/cusadi/blob/dev/Amber/amber_walking_plotting.gif, 2025, accessed: 2025-09-25.
- [11] T. Koolen, T. D. Boer, J. Rebula, A. Goswami, and J. Pratt, “Capturability-based analysis and control of legged locomotion, part 1: Theory and application to three simple gait models,” *Int. J. Robotics Research*, vol. 31, no. 9, pp. 1094–1113, 2012.

- [12] A. L. Hof, "The "extrapolated center of mass" concept suggests a simple control of balance in walking," *Human Movement Science*, vol. 27, no. 1, pp. 112–125, 2008.
- [13] J. Engelsberger, C. Ott, M. A. Roa, A. Albu-Schäffer, and G. Hirzinger, "Bipedal walking control based on capture point dynamics," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems*, 2011, pp. 4420–4427.